PrintHub Checkpoint 1

Beginner implementation guide

This guide explains how to build and demonstrate PrintHub's first end-to-end checkpoint in both supported deployment modes:

1. self-hosted/local, using Podman Compose; and

2. AWS, using CDK, ECS, RDS, S3, Cognito, and CloudFront.

What you are building

PrintHub is a web application for preparing and managing 3D-print jobs. In Checkpoint 1, you are not building the slicing or printer workflow yet. You are building the foundation that later checkpoints use:

```
Browser → Frontend → API → Database
                     ■■■→ Object storage
                     ■■■→ Identity provider
```

The checkpoint is complete when a user can log in, the API can prove that its dependencies are available, the first database schema exists, and one user cannot read another user's data.

Important repository note

The current repository contains the Vue frontend and documentation. It does not yet contain the backend, container definitions, or AWS CDK application. The guide therefore describes the implementation you need to add; some commands will not work until those files exist.

Checkpoint 1 definition of done

You must demonstrate all of the following for each deployment:

- The frontend loads.
- A user can log in.
- `GET /health/live` returns `200`.
- `GET /health/ready` returns `200` only after dependencies are available.
- The initial database migration has run.
- A user can access their own resource.
- A different user receives `403` or `404` for that resource.

AWS also requires a budget alarm and a test notification.

Words you need to know

Term	Meaning
API	The backend HTTP service used by the frontend.
Database	PostgreSQL, where users and application records are stored.
Object storage	Storage for files; SeaweedFS locally and S3 on AWS.

Term	Meaning
Identity provider	The service that handles login; Keycloak locally and Cognito on AWS.
Migration	A versioned change that creates or updates database tables.
Readiness	Whether the service is ready to receive real requests.
Ownership check	A rule that only allows a resource's owner to access it.

Part 1: Build the shared application pieces

Step 1 — Create a backend directory

Add a backend application next to Frontend:

```
Backend/
  app/
    main.py
    config.py
    db.py
    auth.py
    health.py
    models/
    migrations/
  requirements.txt
  Containerfile
```

Use FastAPI, PostgreSQL, a migration tool such as Alembic, and a JWT/OIDC library.

Step 2 — Create the first database tables

Start with a small schema:

```
users
  id
  email
  created_at

resources
  id
  owner_id
  name
  created_at
```

resources.owner_id is the important field. Every protected query must filter by both the resource ID and the authenticated user's ID.

Create and run the first migration:

```
alembic revision -m "create initial schema"
alembic upgrade head
```

Step 3 — Add the health endpoints

Implement:

```
GET /health/live
GET /health/ready
```

/health/live only checks that the API process is running.

/health/ready checks PostgreSQL and object storage. It should return a failure response if either dependency cannot be reached.

Step 4 — Add authentication and ownership protection

Implement:

```
GET /api/v1/me
GET /api/v1/resources/{resource_id}
```

The API should:

1. read the bearer token;

2. validate it against the identity provider's signing keys;

3. identify the user;

4. query resources using WHERE id = :resource_id AND owner_id = :user_id.

Do not fetch a resource by ID alone and check ownership afterward. Filtering in the query prevents accidental data leakage.

Step 5 — Add a signed object-storage check

Configure the API to connect to object storage and generate a short-lived signed upload URL. Do not make the storage bucket public.

Part 2: Self-hosted deployment

Prerequisites

Install:

- Linux, macOS, or a Linux virtual machine;
- Podman 5 or newer;
- Podman Compose support;
- at least 16 GB RAM recommended;
- enough disk space for PostgreSQL, SeaweedFS, and logs.

Step 1 — Add a Compose file

Create a self-hosted Compose file containing:

```
postgres
keycloak
seaweedfs
api
frontend
caddy
```

Only Caddy should be exposed to the host. The other services should communicate over the private Compose network.

Step 2 — Create local configuration

Create .env from .env.example and keep it out of Git:

```
POSTGRES_DB=printhub
POSTGRES_USER=printhub
POSTGRES_PASSWORD=change-me
DATABASE_URL=postgresql+psycopg://printhub:change-me@postgres:5432/printhub
S3_ENDPOINT=http://seaweedfs:8333
S3_BUCKET=printhub-artifacts
```

Also add the Keycloak realm, client ID, frontend URL, and JWT issuer settings.

Step 3 — Start dependency services

```
podman compose up -d postgres keycloak seaweedfs
podman compose ps
```

Do not continue until PostgreSQL, Keycloak, and SeaweedFS are running.

Step 4 — Configure Keycloak

Create a realm named printhub and a public client named printhub-frontend.

Enable:

- Authorization Code flow;
- PKCE with `S256`;
- redirect URI matching your frontend URL;
- the frontend URL as an allowed web origin.

Create two test users: checkpoint-user-a and checkpoint-user-b.

Step 5 — Run migrations and start the application

```
podman compose run --rm api alembic upgrade head
podman compose up -d api frontend caddy
podman compose ps
```

Step 6 — Test self-hosted health

```
curl -i http://localhost/health/live
curl -i http://localhost/health/ready
```

Both should return 200. If readiness fails, inspect:

```
podman compose logs --tail=100 api
podman compose logs --tail=100 postgres
podman compose logs --tail=100 seaweedfs
```

Step 7 — Test self-hosted ownership

1. Open the local frontend.
2. Log in as user A.
3. Create or retrieve a resource owned by user A.
4. Log out.
5. Log in as user B.
6. Request user A's resource.
7. Confirm the response is 403 or 404.

Save the health responses, migration output, successful login, and ownership rejection as evidence.

Part 3: AWS deployment

Prerequisites

You need:

- an AWS account with billing access;
- a selected AWS region;
- AWS CLI credentials;
- Python;

- AWS CDK v2;
- a domain or temporary CloudFront URL;
- permission to create VPC, ECS, RDS, S3, Cognito, Secrets Manager, CloudWatch, and budget resources.

Step 1 — Create the CDK application

Add:

```
infrastructure/
  app.py
  requirements.txt
  stacks/
    network_stack.py
    data_stack.py
    identity_stack.py
    compute_stack.py
    frontend_stack.py
    budget_stack.py
```

Install dependencies and bootstrap CDK:

```
cd infrastructure
python -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
cdk bootstrap aws://ACCOUNT_ID/REGION
```

Step 2 — Create the network and data services

Create a VPC with public and private subnets across two availability zones. Then create:

- RDS PostgreSQL 16 in private subnets;
- a private S3 bucket;
- Secrets Manager entries for database credentials;
- security groups that do not expose RDS publicly.

The API task needs access to RDS and S3 through its task role.

Step 3 — Create Cognito

Create a user pool and public app client with Authorization Code + PKCE. Configure the exact frontend callback and logout URLs.

Create the same two test users:

```
checkpoint-user-a
checkpoint-user-b
```

Step 4 — Build and publish the API

Create an ECR repository, build the API image, and push it:

```
aws ecr get-login-password --region REGION \
  | docker login --username AWS --password-stdin ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com

docker build -t printhub-api ./Backend
docker tag printhub-api:latest ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/printhub-api:latest
docker push ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/printhub-api:latest
```

Step 5 — Deploy ECS and the API load balancer

Create:

- an ECS cluster;
- a Fargate task definition;

- an API service;
- an Application Load Balancer;
- a target group;
- CloudWatch logs;
- an IAM task role.

Set the load balancer health check to:

```
/health/ready
```

Run the initial migration as a one-off ECS task before sending traffic to the service.

Step 6 — Deploy the frontend

Build the Vue frontend with the public Cognito and API settings:

```
cd Frontend
npm ci
npm run build
aws s3 sync dist/ s3://FRONTEND_BUCKET/
```

Put CloudFront in front of the bucket and configure SPA fallback to index.html.

Step 7 — Create the budget alarm before testing

Create an AWS Budget with:

```
Monthly amount: $20
Notifications: 50%, 80%, 100%
```

Confirm the notification subscription and capture a test notification.

Step 8 — Deploy and test AWS

```
cd infrastructure
. .venv/bin/activate
cdk synth
cdk diff
cdk deploy --all
```

Test the API:

```
curl -i https://api.example.com/health/live
curl -i https://api.example.com/health/ready
```

Then repeat the login and ownership test using the CloudFront frontend URL.

Evidence checklist

Both deployments

- ■ Frontend URL opens.
- ■ User A can log in.
- ■ `/health/live` returns `200`.
- ■ `/health/ready` returns `200`.
- ■ Initial migration completed.
- ■ User A can access their own resource.
- ■ User B receives `403` or `404` for user A's resource.
- ■ Object storage is private.

Self-hosted

- ■ `podman compose ps` output.
- ■ Container logs for the API and dependencies.
- ■ Keycloak login evidence.

AWS

- ■ CDK deployment output.
- ■ Healthy ECS task and load-balancer target.
- ■ Private RDS and S3 configuration.
- ■ Budget alarm configuration.
- ■ Budget test notification.

Common beginner mistakes

- • Starting the API before PostgreSQL is healthy.
- • Treating `/health/live` and `/health/ready` as the same check.
- • Exposing PostgreSQL or S3 publicly.
- • Putting database passwords in the Vue frontend.
- • Forgetting to run migrations.
- • Configuring an identity-provider redirect URL that does not exactly match the browser URL.
- • Checking ownership in frontend code instead of enforcing it in the API query.
- • Deploying AWS resources before creating the budget alarm.

After Checkpoint 1

Only begin Checkpoint 2 after both deployments pass this guide. Checkpoint 2 adds profile uploads, model uploads, signed transfers, slicing, G-code generation, retries, and dead-letter handling.